



K.R. Mangalam University

School of Engineering and Technology

Task -2 Prepare a Complexity Comparison Table

Design & Analysis of Algorithms

Course Code - ENCA351

(2026-27)

Submitted by:

Diksha Goel
2401201173
BCA (AI & DS) - A

Submitted To:

Dr. Aarti Sangwan
Faculty, SOET

The table compares the theoretical time and space complexity of each implemented algorithm with the values observed by the benchmarking engine. The observed slope is the gradient of $\log(\text{time})$ against $\log(n)$; it should be close to the exponent k of the theoretical $O(n^k)$ bound.

Algorithm	Theoretical time	Theoretical space	Input range tested	Time at max n (ms)	Ops at max n	Peak mem at max n (KB)	Observed slope (time)	Growth trend observed
Linear Search	$O(n)$	$O(1)$	1,000 – 1,000,000	24.9113	1,000,000	0.16	1.01 (theory 1)	Linear: time and cost are direct proportion
Binary Search	$O(\log n)$	$O(1)$	1,000 – 1,000,000	0.0031	19	0.18	0.08 (theory ~ 0 (log))	Logarithmic: $n \times 10$ comparisons (need)
Jump Search	$O(\sqrt{n})$	$O(1)$	1,000 – 1,000,000	0.0697	1,000	0.26	0.47 (theory 0.5)	Square-root: between logarithmic (need)
Iterative	$O(n)$ mults*	$O(1)$ aux	50 – 2,000	0.5665	1,999	5.17	1.41 (theory 1)	Linear in operation count; linear (big-integer)
Recursive	$O(n)$ mults*	$O(n)$	50 – 2,000	1.0148	1,999	55.11	1.28 (theory 1)	Linear ops; call stack (depth = n)
Memoized Recursive	$O(n)$ mults*	$O(n)$	50 – 2,000	1.2492	1,999	2,477.52	1.27 (theory 1)	Linear ops; stack $O(n)$; cache holds even
Built-in math.factorial	$O(1)$ call*	$O(1)$ aux	50 – 2,000	0.1029	1	13.18	1.51 (theory ~ 0 (const))	Single C-level call; integer cost remains
Single Loop	$O(n)$	$O(1)$	1,000 – 500,000	22.7978	500,000	0.23	1.01 (theory 1)	Linear
Nested Loop (n^2)	$O(n^2)$	$O(1)$	50 – 1,000	40.3517	1,000,000	0.34	2.05 (theory 2)	Quadratic: doubling work
Dependent Loop ($n^2/2$)	$O(n^2)$	$O(1)$	50 – 1,000	18.9793	499,500	0.34	2.07 (theory 2)	Quadratic with constant iterations)
Triple Loop (n^3)	$O(n^3)$	$O(1)$	10 – 120	76.2457	1,728,000	0.28	3.05 (theory 3)	Cubic: doubling n requires 8
Logarithmic Loop	$O(\log n)$	$O(1)$	1,000 – 1,000,000,000,000,000,000	0.0034	60	0.16	0.05 (theory ~ 0 (log))	Logarithmic: i doubling n iterations)

* For factorial, the operation count is exactly $n - 1$ multiplications for every implementation, but each multiplication works on a growing big integer, so measured time grows a little faster than linearly (observed slopes 1.3 – 1.5).

Observed slope = gradient of $\log(\text{time})$ vs $\log(n)$; a slope of k means $\text{time} \propto n^k$. Logarithmic algorithms show slopes close to 0 on a log-log plot.

Growth trend: operation counts versus theory

The instrumented operation counts match the theoretical formulas exactly, which is stronger evidence than timing alone because counts are not affected by machine noise.

Algorithm	Theoretical operations $f(n)$	Ops at max n	Ops ÷ $f(n)$	Observed ops slope	Observed space behaviour
Linear Search	n	1,000,000	1.000	1.00	Constant / small: 0.16 KB at max n
Binary Search	$\lfloor \log_2 n \rfloor + 1$	19	0.950	0.10	Constant / small: 0.18 KB at max n
Jump Search	$\sqrt{n}$	1,000	1.000	0.50	Constant / small: 0.26 KB at max n
Iterative	$n - 1$	1,999	1.000	1.00	Constant / small: 5.17 KB at max n
Recursive	$n - 1$	1,999	1.000	1.00	Stack depth 2,000; 55.11 KB at max n
Memoized Recursive	$n - 1$	1,999	1.000	1.00	Stack depth 2,000; 2,477.52 KB at max n
Built-in math.factorial	1	1	1.000	0.00	Constant / small: 13.18 KB at max n
Single Loop	n	500,000	1.000	1.00	Constant / small: 0.23 KB at max n
Nested Loop (n^2)	n^2	1,000,000	1.000	2.00	Constant / small: 0.34 KB at max n
Dependent Loop ($n^2/2$)	$n(n-1)/2$	499,500	1.000	2.01	Constant / small: 0.34 KB at max n
Triple Loop (n^3)	n^3	1,728,000	1.000	3.00	Constant / small: 0.28 KB at max n
Logarithmic Loop	$\lfloor \log_2 n \rfloor$	60	1.000	0.05	Constant / small: 0.16 KB at max n

Reading the table

- $\text{Ops} \div f(n) = 1.000$ for every algorithm (0.95 for binary search because $\lfloor \log_2 n \rfloor + 1$ is rounded in the worstcase path): the implementations perform exactly the work the theory predicts.
- Time slopes agree with the theory for search and loop programs (within about 0.1).
- Factorial time slopes are above 1 even though operation counts are exactly linear, because big-integer multiplication cost grows with the size of the numbers (see Task 3 observations).
- Only the recursive and memoized factorials use non-constant auxiliary space ($O(n)$ stack depth; the memoized cache is the largest at about 2.4 MB for $n = 2000$).